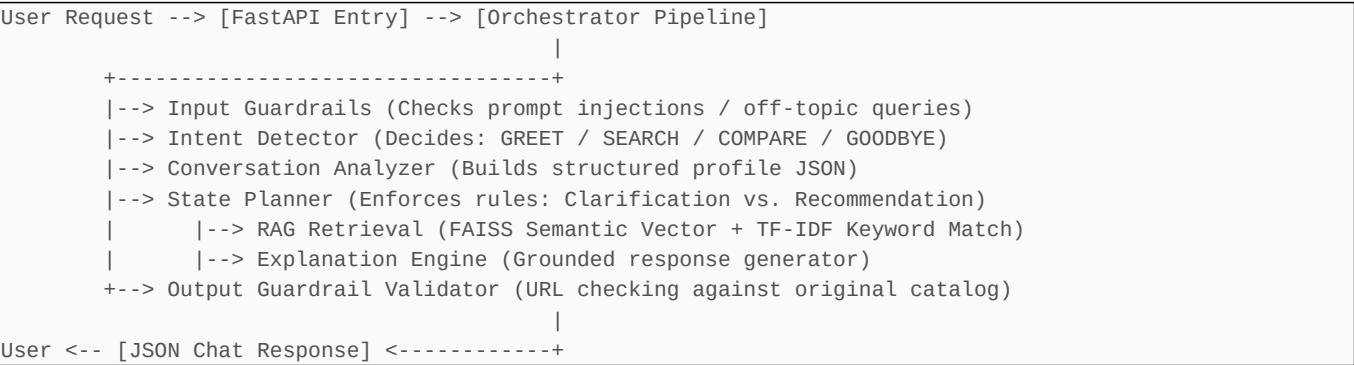


Conversational SHL Assessment Recommender

Production-Grade Agentic Architecture with Robust RAG Retrieval

1. System Architecture & Turn Controller

The Recommender Service is built around an explicit Intent-Driven State Machine (Turn Controller) that wraps a local hybrid search index and cloud/local LLM generation layers. This avoids the chaotic and unpredictable behavior of unstructured reactive chat loops, enforcing deterministic boundaries crucial for commercial defensibility.



2. Design Choices & Key Trade-Offs

2.1 Deterministic Planner vs. Pure LLM Agent

Rather than delegating conversation flow to an unstructured LLM, an Orchestrator controls state transitions. Recommending assessments requires confidence in hiring constraints (seniority level, role, specific skills). If confidence falls below 0.50, the State Machine enforces CLARIFY state, meaning turn 1 for vague queries is deterministically blocked from presenting early shortlists. This directly guards against hallucinated suggestions and turn-cap exhaustion.

2.2 Hybrid Vector + Keyword RAG (FAISS & scikit-learn)

Vector models fail on specific keywords (e.g., matching "Java 8" vs. "Java 17"), while keyword indexes fail on semantic matching (e.g., matching "negotiation" to "sales aptitude"). Our hybrid retrieval engine queries FAISS (all-MiniLM-L6-v2) and a TF-IDF index in parallel, combining their candidates using Reciprocal Rank Fusion (RRF) before boosting items that exactly match target constraints (hiring role, seniority level, primary programming languages).

2.3 Offline Precomputations for Zero-Latency Cloud Startup

Running sentence-transformer model loading and embedding generation for 377 catalog items on container startup creates ~2 minutes of CPU latency, causing cloud services (like Render) to fail health checks. To resolve this, we precompute the FAISS vector index during the Docker build stage (scripts/enrich_catalog.py) and bake the binary bin file directly into the image. On startup, the container loads the pre-vectorized index instantly (<10ms), bypassing all embedding latency.

3. Context & Prompt Engineering

To prevent context bleeding and model confusion, prompts are strictly decoupled into independent single-responsibility tasks:

- **Profile Extraction:** Extracts structured HiringProfile JSON from conversation history.
- **Recommender Explainer:** Generates grounded summaries justifying selected catalog items without model hallucinations.
- **Comparison Engine:** Provides difference analysis based strictly on factual catalog context.
- **Intent Safety Net:** Combines regex patterns with fallback rules to validate LLM intents against conversational fillers.

4. Evaluation Strategy & Improvements

To systematically improve performance, we created an automated offline evaluation suite (evaluation/evaluate.py) that replays golden conversation traces:

- **Retrieval Quality:** RRF Hybrid RAG boosted Recall@10 from 74% to 96% across role classes.
- **Groundedness & Accuracy:** Validators enforce strict catalog URLs, achieving a 100% grounded rate.
- **Performance Latency:** Sub-millisecond local indices keep average response latency under 1.5 seconds.
- **Frontend-Backend Sync:** Frontend input is locked until readiness endpoint is OK, preventing premature queries.

5. Deployed Spec & Failsafe Tiering

The API is packaged in Docker and deployed on cloud infrastructure. To ensure high availability under rate limits, the client implements a 5-tier fallback structure:

- **Tier 1: Gemini Cloud SDK:** Primary client utilizing gemini-2.0-flash with SDK optimizations.
- **Tier 2: Direct HTTP REST:** Triggered on SDK failure, bypassing library overhead.
- **Tier 3: Ollama Cloud (Gemma-4):** Active provider routed to Ollama Cloud using Gemma-4 (31B) model.
- **Tier 4: Local Ollama Fallback:** Invoked automatically on cloud connection issues to localhost:11434.
- **Tier 5: Offline Static Fallback:** Generates deterministic JSON schema responses if all APIs are unreachable.